Resilient University Information System

Team **Aibyn Talgatov (CSE-2505M), Gani Uapov (CSE-2502M)** Repository **github.com/a169n/fault_tolerance_endterm**

Full report **reports/endterm/REPORT.md**

10/10
scenarios passed by the endterm, against 6 (midterm) and 2 (baseline)

90
controlled runs: 10 scenarios × 3 modes × 3 repetitions, none aborted

0
duplicate or lost payments in midterm and endterm; the baseline double-charged 11 058 times

63/63
attacks per E9 run denied and audited in endterm; 0 denied in the other modes

1 Introduction and relationship to the midterm

The midterm built a university platform (students, payments, transcripts, timetables) with software fault tolerance: retries with backoff, timeouts, a circuit breaker, health checks, idempotent payments, checkpoint and rollback, graceful degradation, replicas and a database standby. The endterm extends the same system, as the assignment requires, with four new layers: cloud/microservice resilience, observability with alerting, security with intrusion tolerance, and systematic verification by fault injection.

One codebase, three modes. Every container runs the same image; two switches select the behaviour. A measured difference between modes is therefore caused by the mechanisms, not by different code.

MODE	SWITCHES	BEHAVIOUR
baseline	`FT=0` `ENDTERM=0`	Same replicas and standby, but nothing uses them: no retries, timeouts, idempotency or security.
midterm	`FT=1` `ENDTERM=0`	The midterm mechanisms S1–S10 and H1–H5, unchanged (tag `midterm-final` = commit `51100cf`).
endterm	`FT=1` `ENDTERM=1`	Midterm plus redundant gateways, outlier ejection, JWT authentication, authorization, validation, rate limiting and audit.

Monitoring (metrics, structured logs, correlation IDs, Prometheus, Grafana, alerts) and credential hygiene are on in all three modes, so all modes are measured the same way. Deploy with `npm run up:baseline | up:midterm | up:endterm`.

2 Requirements, assumptions and threat model

Each requirement can be checked by an experiment or a test. The thresholds are fixed before the campaign and applied unchanged to all three modes.

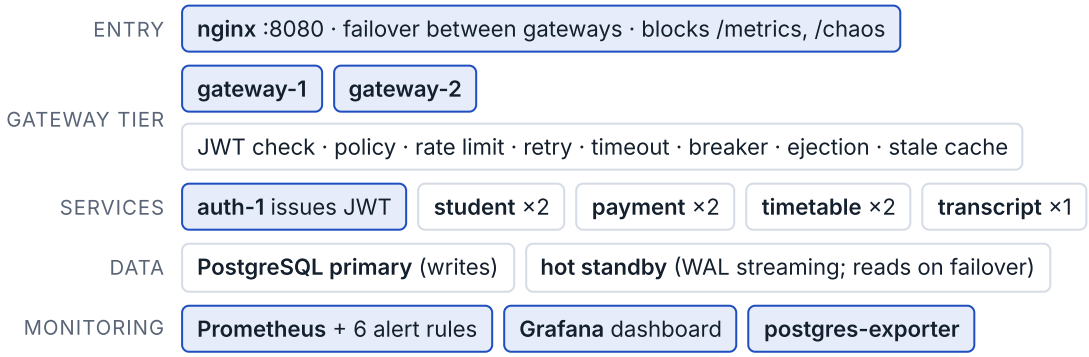
#	REQUIREMENT (ENDTERM TARGET)	CHECKED BY
DR1	A crashed or killed replica keeps request availability ≥ 99.9 %, no lost or duplicate payment	E1, E4
DR2	The gateway is not a single point of failure: availability ≥ 99.5 % when one gateway dies	E7
DR3	A slow replica is taken out of rotation: goodput during the fault ≥ 70 % of normal, p95 ≤ 1 s	E3
DR4	Primary DB down: reads ≥ 99 % available, outage alerted within 10 s	E2
DR5	An interrupted payment leaves no duplicate, lost or orphaned record; the ledger balances	E5
DR6	At 110 clients: error rate ≤ 1 %, p95 ≤ 1 s, legitimate traffic never rate-limited	E6
DR7	Each incident class is detected by its alert, and no unexpected alert fires	E2, E4, E7–E10
DR8	Every unauthenticated or over-privileged request is denied (401/403) and audited; no data changes	E9
DR9	Malformed input gets a 4xx, never a 5xx; floods and brute force are rate-limited; nothing crashes	E10
DR10	Every request is traceable across all hops by one correlation ID that cannot inject into logs	unit + integration tests

Assumptions. Crash, omission and timing faults (no Byzantine faults); one Docker host; synthetic data only; the host, Docker daemon and pinned images are trusted; secrets are generated per install into a git-ignored `.env`.

Threat model (STRIDE over five trust boundaries; full version in `THREAT-MODEL.md`). In scope: anonymous clients, students exceeding their rights, a stolen student token (limited by its 15-minute lifetime and the student's own rights), and faulty components. Out of scope: a malicious insider with root on the host.

BOUNDARY	MAIN THREATS	MAIN CONTROLS
Client → nginx	missing/forged/expired/ `alg:none` tokens, brute force, malformed input, floods, replayed payments, log injection	JWT verification, authorization matrix, validation, rate limits, required `Idempotency-Key`, request-ID sanitising
nginx → gateways	gateway as SPOF, double charge when a POST is re-sent	two gateways; client-supplied idempotency key
Gateways → services	direct calls from a compromised container, chaos-endpoint abuse, auth outage	service-side validation, chaos token, local token verification
Services → PostgreSQL	`trust` logins, SQL injection, destructive SQL, primary down	scram-sha-256, parameterised SQL, role without DELETE/TRUNCATE/DDL, standby reads
Host	committed secrets, exposed ports, vulnerable dependencies	generated secrets, loopback-only ports, pinned versions, `pip-audit`

3 Cloud/microservices architecture and resilience design



16 containers on one Docker host. Blue boxes are new in the endterm.

Inherited from the midterm

- Timeout 800 ms per attempt; up to 3 attempts (2 retries) with full-jitter exponential backoff (base 50 ms), each on the next replica.
- Circuit breaker per pool: opens after 5 failures, half-opens after 5 s.
- Health checks every 1 s take dead replicas out of rotation.
- Idempotent payments (unique key) and a checkpoint + rollback sweep for interrupted payments (after 10 s).
- Graceful degradation: a failed read is served from a stale cache (≤ 60 s old) as HTTP 203; reads fail over to the standby.
- Restart policy (self-healing) and storage page checksums.

New in the endterm

- **Redundant gateways.** nginx re-sends a failed request to the other gateway. This is safe only because every payment must carry a client `Idempotency-Key` (otherwise 400).
- **Outlier ejection.** A replica with 3 consecutive failures leaves rotation for 10 s; never more than half a pool.
- **Liveness vs readiness.** `/health/live` (process up) and `/health/ready` (dependencies up); Docker healthchecks every 2 s.
- **Auth outage degrades gracefully.** Tokens are verified locally at the gateway, so an auth outage blocks only new logins.

Remaining single points of failure (accepted): nginx, the PostgreSQL primary (no automatic failover), `auth-1`, `transcript-1`, `postgres-exporter` (the only observer for `DatabaseDown`) and the host itself. In Kubernetes the same design maps to Deployments with liveness/readiness probes, a Service per pool and outlier detection in a service mesh (discussion only; not deployed).

4 Monitoring, logging, metrics and alerting

Structured logs. Every service writes one JSON line per event with `ts, svc, mode, kind, rid, level`.

Correlation ID. The client's `X-Request-ID` (nginx generates one if it is missing; a service replaces an unsafe one with a fresh UUID) travels through nginx, the gateway and every service and appears in every log line and response. Hostile IDs (newlines, 10 KB, non-ASCII) are replaced, so they cannot inject into logs.

Metrics (Prometheus, scraped every 2 s): request count and latency histogram per route, upstream attempts by instance and outcome, instance up/ejected, breaker state, auth denials, rate-limited requests, and payments stuck in `pending`. Route labels use templates, not raw paths, so the number of series stays bounded.

Dashboard. One provisioned Grafana dashboard with 13 panels; `OBSERVABILITY.md` walks one real incident (E2) from alert to log lines by request ID.

ALERT	THRESHOLD	EXPECTED RESPONSE
InstanceDown	target down for 4 s	restart policy heals; investigate after 60 s
DatabaseDown	`pg_up == 0` for 4 s	reads use standby; restore primary
HighErrorRate	gateway 5xx share > 5 % (15 s) for 6 s	check failing instances and breaker
HighLatencyP95	gateway p95 > 1 s (15 s) for 6 s	look for slow or ejected instances
AuthFailureSpike	> 2 denials/s (30 s) for 4 s	possible attack: review audit by rid/sub
PaymentsStuckPending	pending > 15 s for 10 s	confirm rollback sweep; reconcile

All six rules have `promtool` unit tests. Measured mean detection delay in the endterm runs (injection → alert firing):

ALERT (SCENARIO)	DETECTION
DatabaseDown (E2)	8.3 s
InstanceDown (E4)	6.0 s
InstanceDown (E7)	6.3 s
HighErrorRate (E8)	10.3 s
AuthFailureSpike (E9)	8.3 s
AuthFailureSpike (E10)	9.0 s

5 Security controls and intrusion-tolerance strategy

CONTROL	HOW IT WORKS	PROOF
Authentication	Auth service checks scrypt password hashes and issues HS256 JWTs (15 min; issuer and audience checked; only HS256 accepted, so `alg:none` fails)	unit tests; E9
Authorization	Role × route matrix at the gateway (student / staff / admin). A student reads and pays only for themselves. Any error → deny (fail closed)	32-case matrix test; E9
Input validation	Strict pydantic models (ID patterns, 0 < amount ≤ 10 000, 2 decimals, no extra fields); body ≤ 16 KiB; safe path segments	unit tests; E10
Rate limiting	Token buckets: login 1/s burst 5 per IP; student 10/s burst 20; staff 5 000/s. Over quota → 429 with `Retry-After`. Buckets are per gateway	unit tests; E10
Audit logging	Every deny, payment, admin action and login is logged with request ID, never with a password or token	secret-leak test; E9
Secrets and least privilege	Generated secrets in `.env`; scram-sha-256; app DB role without DELETE, TRUNCATE or DDL; internal ports on loopback; `/chaos` needs a token	integration tests
Dependency checks	`pip-audit` (the CI `deps` job's command) found 24 known vulnerabilities (17 in PyJWT 2.10.1, 7 in Starlette 0.41.3); fixed by upgrading to PyJWT 2.15.1 and FastAPI 0.162.2 / Starlette 1.10.0; audit then clean	before/after reports

Intrusion tolerance. The design assumes some attacks will get through a layer and limits the damage: a stolen token works only for 15 minutes and only within one student's rights; an attack burst is denied, counted and alerted (`AuthFailureSpike`); the database role cannot destroy data; a flood is cut by rate limits before it reaches the services; nginx failover cannot double-charge because of the client key.

Final review fixes (after the campaign): an idempotency key could be reused across students (now 409); a `{param}` route could reach a service's own `/openapi.json`; the stale cache kept 404s without a size bound; the app role could update user roles. Each fix has tests and was re-checked with one endterm run each of E5, E7 and E9.

6 Verification and validation plan

LEVEL	WHAT	TESTS
Unit (Python)	every mechanism, policy matrix, tokens, validation, limiter, metrics	409
Harness (Node)	metrics, acceptance, alert correctness, aggregation	95
Alert rules	`promtool test rules`	6 rules
Integration	against a live endterm stack: login, idempotent payment, denial, 422 not 5xx, rid across services, DB role limits	17
System	10 fault-injection and security experiments with acceptance criteria	90 runs

Acceptance criteria are machine-checked predicates over each run's measurements (for example "request availability ≥ 99.9 %", "no duplicate charge", "DatabaseDown fires within 10 s"). A scenario passes only if every criterion passes in every repetition.

CI (GitHub Actions): `unit`, `harness`, `config` (compose for 3 modes, promtool, `nginx -t`), `deps` (pip-audit), `integration` (full stack in Docker). All five jobs pass on `main` (run 42787494892 in `a169n/fault_tolerance_endterm`).

7 Fault-injection and security-test methodology

Each run lasts 70 s under a closed-loop workload of 10 clients (60 % student reads, 20 % transcripts, 20 % payments; 20 % of payments are re-sent with the same key). The fault is injected at +20 s and repaired at +45 s. Before every run the harness restores a known state: all containers healthy and replicating, chaos settings reset, payments truncated, balances zeroed, and no alert firing. Each run records every request (with its request ID), all service events, alerts (polled every 1 s), CPU and memory, and the payments table for key-level integrity checks.

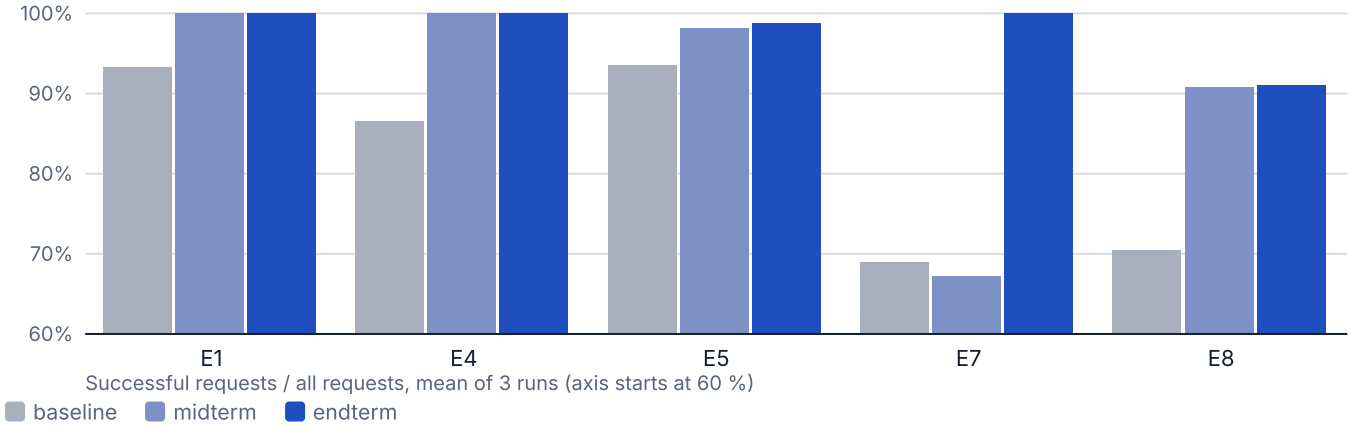
CODE	SCENARIO	INJECTION
E1	Service process crashes	student-1 kills itself (/chaos/crash)
E2	Primary database down	docker compose stop postgres-primary
E3	One replica becomes slow	+3 000 ms latency on student-1
E4	A whole node dies	kill student-1 and payment-1 together
E5	Payment interrupted mid-way	both payment replicas crash after the checkpoint
E6	Load spike	+100 concurrent clients (10 to 110)
E7	One gateway dies	docker compose kill gateway-1
E8	Error spike (alert test)	failRate 1.0 on both student and both payment replicas
E9	Unauthorized access	63 attacks in 7 kinds: no / forged / expired / alg:none token; student pays for another, creates a student, lists all
E10	Abuse and malformed input	203 malformed bodies, request flood, 50 bad logins

E1–E7 and E8 are fault or monitoring experiments (8 ≥ 4 required); E9 and E10 are security experiments (2 ≥ 2 required). 3 modes × 3 repetitions each = 90 runs, executed in mode order from a fresh database volume per mode. Hypothesis, steps, expected result and limitations of each experiment: `TEST-PLAN.md` (E1–E10); observed results per experiment: full report §8.5.

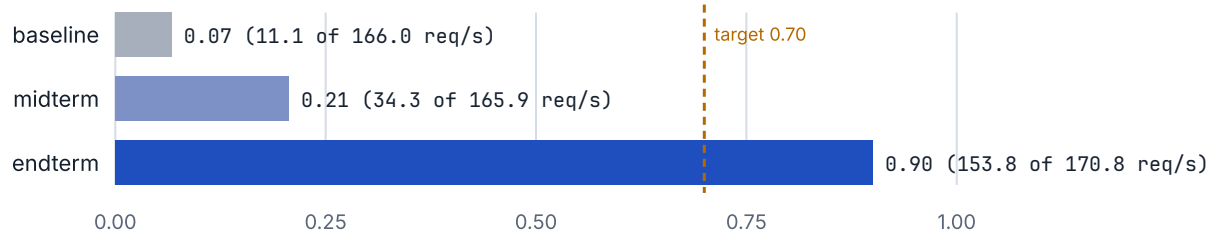
8 Results: baseline vs midterm vs endterm

CODE	SCENARIO	TYPE	BASELINE	MIDTERM	ENDTERM
E1	Service process crashes	fault	FAIL 0/3	PASS 3/3	PASS 3/3
E2	Primary database down	fault	FAIL 0/3	PASS 3/3	PASS 3/3
E3	One replica becomes slow	fault	FAIL 0/3	FAIL 0/3	PASS 3/3
E4	A whole node dies	fault	FAIL 0/3	PASS 3/3	PASS 3/3
E5	Payment interrupted mid-way	fault	FAIL 0/3	PASS 3/3	PASS 3/3
E6	Load spike	fault	PASS 3/3	PASS 3/3	PASS 3/3
E7	One gateway dies	fault	FAIL 0/3	FAIL 0/3	PASS 3/3
E8	Error spike (alert test)	monitoring	PASS 3/3	PASS 3/3	PASS 3/3
E9	Unauthorized access	security	FAIL 0/3	FAIL 0/3	PASS 3/3
E10	Abuse and malformed input	security	FAIL 0/3	FAIL 0/3	PASS 3/3
	Scenarios passed		**2 / 10**	**6 / 10**	**10 / 10**

"k/3" = repetitions in which every criterion passed. E6 and E8 pass in the baseline because their criteria do not discriminate: at 110 clients nothing failed because of load and "not rate-limited" is trivially true without a limiter (E6), and the alerting that E8 tests is on in every mode (see §9).



Request availability. Replica faults (E1, E4, E5) are masked from the midterm on; a dead gateway (E7) is masked only by the endterm's second gateway. E8 stays near 91 % in FT modes because failed payment writes cannot be served from the cache.



E3, slow replica: goodput during the fault as a share of goodput before it. Only per-instance outlier ejection (endterm) keeps the system above the 0.70 target.

E7 (one gateway killed): availability 68.97 % (baseline), 67.16 % (midterm), **99.99 %** (endterm).

E9: in baseline and midterm all 63 attacks per run succeeded (in run r1, 41 attacker payments and 9 fake students reached the database). In endterm all 63 were denied (36 × 401, 27 × 403), 63 audit events were written and `AuthFailureSpike` fired.

E10: malformed payments caused 20 (baseline) and 10 (midterm) server errors in run r1; endterm answered all 203 with 4xx. A 1 000-request flood got 936 × 429; of 50 brute-force logins only 10 reached a password check (5 per gateway,

two gateways) and 40 got 429 (run r1). Legitimate traffic was never rate-limited in any of the 90 runs.

Key measurements (mean of 3 runs)

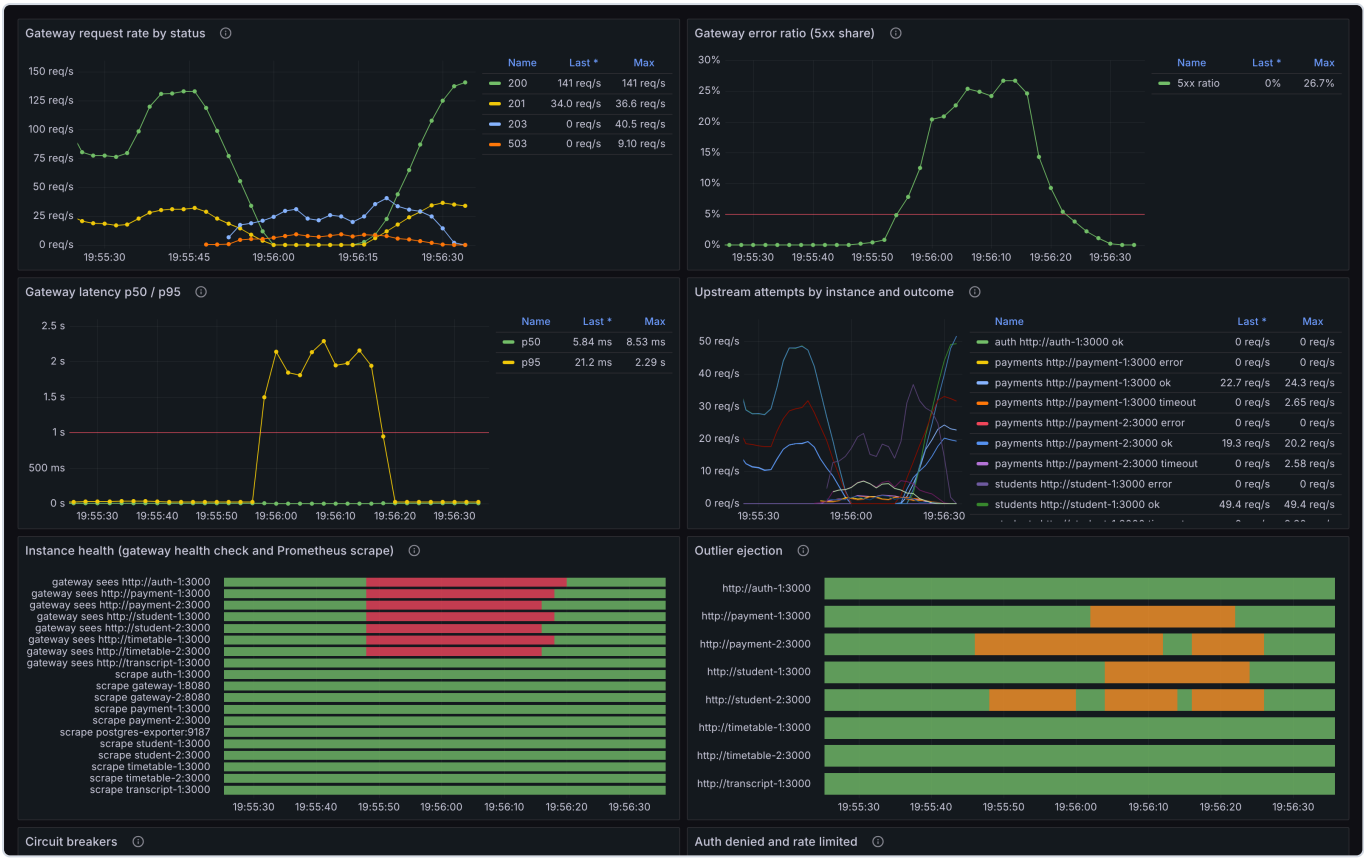
CODE	REQUEST AVAILABILITY			P95 LATENCY, MS			DUPLICATE CHARGES (3 RUNS)		
	BASE	MID	END	BASE	MID	END	BASE	MID	END
E1	93.22 %	99.99 %	99.99 %	24	27	24	1064	0	0
E2	99.30 %	97.22 %	96.70 %	24	30	24	785	0	0
E3	99.99 %	99.98 %	99.98 %	29	36	25	859	0	0
E4	86.48 %	99.99 %	99.99 %	26	27	26	811	0	0
E5	93.45 %	98.18 %	98.76 %	26	25	26	862	0	0
E6	99.53 %	99.57 %	99.59 %	398	395	473	2176	0	0
E7	68.97 %	67.16 %	99.99 %	25	27	25	863	0	0
E8	70.46 %	90.75 %	91.03 %	25	24	20	887	0	0

Data integrity. Midterm and endterm, checked key by key, lost no acknowledged payment and had 0 duplicate charges and no orphaned payments. The baseline does not store the client key, so it could only be checked by count (no shortfall found); the baseline, which ignores the idempotency key, charged 11 058 client re-sends twice and left orphaned `pending` payments in E5 and E10.

Where the endterm is worse or the evidence is weak.

- **Cost:** at the 110-client peak (E6) endterm p95 was 473 ms against 398 / 395 ms, while serving more requests (362.0 req/s against 314.5 / 319.6): the endterm's extra per-request gateway work (token, policy, limiter) shows up as a longer tail, but the second gateway changed at the same time, so the cost cannot be attributed to security alone. Still under the 1 s target.
- **E2 availability:** the baseline scores higher (99.30 % against 96.70 %) only because its clients hang and stop sending; it served 2.1 req/s during the fault against 23.3 for the endterm.
- **DR7 not met:** in E2 `HighLatencyP95` fired in all 3 endterm runs although not expected. It is a true symptom: a replica first waits 1 s for the dead primary before reading from the standby, longer than the gateway's 800 ms attempt timeout. Endterm meets 9 of 10 requirements.

One incident end to end (E2, endterm run r1)



First failed call 0.26 s after the primary stopped; ejection and breaker at 0.33 s; health probe at 0.46 s; `DatabaseDown` pending at 3.6 s and firing at 8.0 s. Reads continued from the standby or the stale cache; writes failed fast with 503 and recovered 5.6 s after the repair (5.2–10.1 s across the three runs). Full walkthrough with log lines: `OBSERVABILITY.md`.

9 Discussion, limitations and lessons learned

Discussion. The endterm's measurable gains over the midterm are exactly where its new mechanisms act: killing a gateway no longer causes an outage, a slow replica no longer costs most of the throughput, and attacks turn from silent successes into denied, audited and alerted events. Everything the midterm already masked (crashes, node loss, interrupted payments) stays masked. The baseline shows why: without idempotency every client re-send became a second charge.

Limitations

- One host: all 16 containers share CPU, disk and the Docker daemon; host failure is untested.
- 3 repetitions: enough to see spread, not enough for significance tests.
- Closed-loop workload flatters a system that hangs (fewer requests, fewer failures).
- Some criteria do not discriminate between modes (E6 and E8 pass in the baseline; E3's whole-run p95; the ledger check cannot see double charges).
- Detection and recovery times include window-close artefacts; per-operation recovery was derived afterwards from raw logs.
- The campaign measured the code before the dependency upgrade and the final review fixes; those were checked with single repetitions, not a re-run.
- Accepted risks: rate-limit state is per gateway (2× quota), no mTLS or public TLS, no token revocation, one shared DB role, containers run as root.

Lessons learned

- Measure availability more than one way: request-based, time-based and goodput disagreed in E2, E3 and E7.
- Check that the baseline fails a criterion before trusting it.
- An "unexpected" alert is a finding: it exposed the timeout-nesting defect in E2.
- Read raw data before believing an aggregate; the correlation ID made that fast.
- Redundancy needs a safe retry contract: the nginx failover is safe only because the client sends the idempotency key.
- Rate limits in a replicated tier must be sized for the whole cluster.

Next steps

Nest the timeouts (DB acquire < gateway attempt) and re-run E2; re-run the campaign on the upgraded dependencies; open-loop load for E3 and E6; shared rate-limit state; a second nginx and automatic DB failover; per-service DB roles and non-root containers.

10 Conclusion and references

In 90 controlled runs the endterm passed all 10 scenarios in every repetition, against 6 for the midterm configuration and 2 for the baseline. It meets 9 of its 10 requirements (DR7 missed by one unexpected but genuine alert). Across the 60 midterm and endterm runs no payment was lost, duplicated or left pending. The weak points are reported with the data that shows them rather than tuned away.

References: endterm and midterm assignments (`docs/requirements/`); Avižienis et al., "Basic Concepts and Taxonomy of Dependable and Secure Computing", IEEE TDSC 2004; Shostack, *Threat Modeling* (STRIDE), 2014; Nygard, *Release It!*, 2018; Beyer et al., *Site Reliability Engineering*, 2016; Schroeder et al., "Open Versus Closed: A Cautionary Tale", NSDI 2006; RFC 7519 / RFC 8725 (JWT); OWASP API Security Top 10 (2023); Prometheus, nginx, Envoy, PostgreSQL 16 and Kubernetes documentation.

11 Appendix

WHAT	WHERE
Full technical report (all tables, per-experiment hypotheses and observations)	`reports/endterm/REPORT.md`
Results tables and raw data	`reports/endterm/RESULTS.md`, `results.csv`, `runs/*/metrics.json`, `logs/endterm/`
Threat model, alerts, test plan, observability, demo	`THREAT-MODEL.md`, `ALERTS.md`, `TEST-PLAN.md`, `OBSERVABILITY.md`, `DEMO.md`
Configuration	`docker-compose.yml`, `config/nginx`, `config/prometheus`, `config/grafana`, `config/postgres`
Code: mechanisms, security, services	`app/fault_tolerance/`, `app/security/`, `app/services/`
Tests and fault injection	`tests/unit`, `tests/integration`, `experiments/` (`npm run exp <scenario>`, `npm run campaign -- --reps 3`, `npm run demo`)

Individual contributions

The commits after `midterm-final` were made by an AI coding assistant working on the team's behalf (full report §11.6). Each team member describes their own part below.

Aibyn Talgatov: ...

Gani Uapov: ...

★ Defense cheat sheet: likely questions

Why three modes instead of comparing against the old midterm commit?

Same image, same host, same harness: the only variable is the switch, so a difference in the numbers is caused by the mechanisms. The old midterm campaign is quoted for reference only.

Why doesn't a failed gateway cause double charges when nginx re-sends the POST?

Every payment must carry a client-generated `Idempotency-Key`; a request without one gets 400. The database has a unique constraint on the key, so the re-sent request returns the stored payment instead of charging again. Across all 30 endterm runs: 0 duplicate charges.

What is the difference between the circuit breaker and outlier ejection?

The breaker is per pool: it stops calling a whole service that keeps failing. Ejection is per instance: it removes one bad replica after 3 consecutive failures for 10 s. A per-pool breaker cannot isolate one slow replica, because every successful retry on the healthy replica resets it (found in the midterm); ejection raised goodput from 0.21 to 0.90 of normal.

Why does the baseline have higher availability than the endterm in E2?

Closed-loop clients: baseline requests hang without timeouts, so clients send almost nothing and record few failures (2.1 req/s served). The endterm fails writes fast and keeps serving reads (23.3 req/s). Time-based availability and goodput show the real picture.

How is an incident detected and diagnosed?

Detection: the first failed call is seen in under 1 s; the alert fires after the 2 s scrape and evaluation, its `for` period and, for rate rules, the window fill (6.0–10.3 s measured in the endterm). Diagnosis: the Grafana panels show which instance and which status; the request ID of a failed request finds every log line on every hop.

What happens if someone steals a student's token?

It expires within 15 minutes and only allows that student's own reads and payments. There is no revocation list; this is accepted risk R3.

Why can't `alg:none` or a forged token pass?

The verifier accepts only HS256 with the server's secret and checks issuer, audience and expiry; anything else raises an error and the gateway fails closed with 401. E9 tested all four token attacks: 36 × 401.

Does security slow the system down?

At normal load no overhead is visible. At the 110-client peak the p95 grew from about 395 to 473 ms while throughput rose. Probably the per-request token, policy and limiter work, but the second gateway changed at the same time, so we cannot attribute it to security alone. It stays below the 1 s target.

Which requirement is not met, and why not fix the criterion?

DR7: `HighLatencyP95` fired unexpectedly in E2. It is a real defect (the 1 s DB wait is longer than the 800 ms gateway timeout). Changing the acceptance rule after seeing the result would make the test meaningless, so it is reported as a miss and listed as the first next step.

How do you know the numbers are not made up?

Every table is generated by `experiments/aggregate.ts` from the 90 committed `metrics.json` files; the security counts and the incident timeline come from per-run `metrics.json` and the committed raw request and event logs. Re-running `npm run campaign -- --reps 3` gives new measurements of the same kind, not identical numbers.

What would you do with more time?

Fix the timeout nesting, re-run the campaign on the upgraded dependencies, use an open-loop load generator, share rate-limit state between gateways, and remove the remaining single points of failure (second nginx, automatic DB failover).